

embed-api

An embedding API for multilingual-e5-large

FastAPI, ONNX Runtime (int8), Docker, GitHub Actions, Kubernetes

The model shapes the API

e5	The API
Trained with <code>query:</code> and <code>passage:</code> prefixes	<code>input_type</code> is required; the server adds the prefix
Reads at most 512 tokens, prefix included	Truncates, and reports <code>tokens</code> and <code>truncated</code> per text
560M parameters, slow on CPU	One inference at a time, off the event loop, with a token budget per request
Output is normalised, 1024 dimensions	Cosine similarity is a dot product
The model repository is 9.5 GB and changes	Pinned revision, only the needed files, revision shown in <code>/v1/info</code>

What the task asked for

- Validation: one pydantic schema drives both validation and the Swagger docs. Body size is checked before parsing, the token budget after tokenising.
- Errors: one error format with a stable `code` and the request id.
- Logging: JSON, one line per request, `X-Request-ID` on everything. Input text is never logged, and a test checks this.
- Code: the model sits behind a small interface, so most tests use a fake model and run in under a second.

Request lifecycle

```
client -> request id and access log
        -> body size limit           (413)
        -> pydantic validation       (422)
        -> model loaded?             (503)
        -> count tokens, budget      (422)
        -> wait for inference slot   (503 after 30 s)
        -> embed in a worker thread -> response
```

- The model loads in the background: liveness passes at once, readiness when the model is loaded
- A request whose client has disconnected is dropped before inference
- Thread counts follow the container's CPU limit, not the host's cores

Faster inference, and what it costs

Backend	Weights	1 text	32 texts	nDCG@10, DanFEVER
PyTorch fp32	2.2 GB	115 ms	1.83 s	0.840
ONNX Runtime fp32	2.2 GB	58 ms	1.82 s	
ONNX Runtime int8	0.56 GB	33 ms	1.04 s	0.828

- ONNX Runtime halves single-text latency; int8 removes another 40%
- int8 loses 1.4% nDCG@10 on DanFEVER and 3.2% on TwitterHjerne (77 queries)
- The fp32 scores match the official `mteb` package
- Quantised for AVX2, which the server's CPU has
- No torch at runtime: the image is 0.6 GB, against 1.8 GB with torch

A bug worth finding

Dynamic int8 quantisation picks its scale from the whole input tensor.

In a batch, a text's embedding depended on the other texts: the same text gave cosine 0.994 to itself embedded alone.

Same text, different vector. That breaks caching and deduplication.

Fix: one text per forward pass (about 10% less throughput), and a test that the result does not depend on the batch.

Shipping it

- Docker: the int8 model is built in a separate stage, so the image has no torch; it runs as non-root with a read-only filesystem
- CI: lint, type check, tests, build, start the container and call it, then push
- Deployment: Kubernetes at home, updated through GitOps (ArgoCD) on every new image

Next steps for production: authentication and rate limiting, metrics, reducing the int8 quality loss, autoscaling, or a dedicated server such as Hugging Face TEI

Demo

1. Swagger at `/docs`
2. A validation error and its request id in the logs
3. `examples/limits.py` : truncation and the token budget
4. `examples/demo.py` : Danish, English and German